ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – СОФИЯ

ФАКУЛТЕТ ПО КОМПЮТЪРНИ СИСТЕМИ И ТЕХНОЛОГИИ

КОМПЮТЪРНО И СОФТУЕРНО ИНЖЕНЕРСТВО

Курсова работа на тема

„Courier Company Database“

Предмет: Нерелационни бази данни

Изготвила: Теодора Димитрова

Факултетен номер: 121222136

Група: 40 “б”

Съдържание

I. Въведение 3
II. Теоретична и технологична обосновка 4
III. Архитектура и реализация 5
Колекция - Customers 6
Колекция Couriers 7
Колекция Offices 8
Колекция Payments 9
Колекция Packages 10
IV. Задължителни CRUD операции 11
Създаване: 11
Четене: 11
Актуализация: 12
Изтриване: 13
V. Допълнителни заявки 14
- Пример за заявка, използваща агрегиране, групиране или сортиране: 14
- Пример за заявка, която комбинира данни от две различни колекции: 14
VI. Контрол на достъпа до данните 16
VII. Използвана литература: 18

I. Въведение

В настоящия проект е разгледан реален бизнес казус – система за управление на куриерска фирма. Основната цел на системата е да съхранява и обработва информация за клиенти, куриери, пратки, офиси и извършени плащания. Такива системи са широко използвани в съвременната логистика и електронната търговия, което прави избрания казус подходящ за моделиране на база данни с практическо приложение.

Основният проблем, който се разглежда, е необходимостта от ефективно съхранение и обработка на разнообразни данни, свързани с пратките. В една куриерска система всяка пратка може да съдържа различна информация – податель, получател, адреси, статус на доставка, история на движенията, тип услуга и начин на плащане.

При използване на релационни бази данни това би довело до създаване на множество таблици и сложни връзки между тях, както и до необходимост от използване на JOIN операции при извличане на информация. Това усложнява както структурата на данните, така и изпълнението на заявките.

За реализацията на проекта е избрана нерелационна база данни MongoDB. Изборът е мотивиран от възможността за съхранение на данни в документо-ориентиран формат, който позволява използване на вложени обекти и масиви. Това прави модела по-гъвкав и по-подходящ за описания бизнес казус.

Допълнително предимство на MongoDB е възможността за ефективна работа с големи обеми от данни и висока производителност при операции по запис и извличане, което е важно за системи с интензивен поток от заявки като куриерските платформи.

II. Теоретична и технологична обосновка

Нерелационните бази данни (NoSQL) предлагат по-гъвкав модел за съхранение на информацията в сравнение с традиционните релационни бази данни. Те не изискват фиксирана схема и позволяват динамично добавяне на нови полета към документите. Това ги прави особено подходящи за системи, при които структурата на данните може да се променя или разширява във времето.

В настоящия проект е използвана документо-ориентирана база данни MongoDB. Данните се съхраняват под формата на JSON документи, което улеснява тяхната обработка и интеграция с уеб приложения. Всеки запис представлява самостоятелен документ, който може да съдържа вложени обекти и масиви.

В куриерската система основният обект е пратката (package), която съдържа различни типове информация – подател, получател, адреси, статус на доставка, както и история на статусите. Тази информация може да бъде съхранена в един документ чрез използване на вложени структури. Например полето *statusHistory* представлява масив от обекти, съдържащи информация за всяка промяна на статуса на пратката.

Документо-ориентираният модел позволява също така различните документи в една и съща колекция да имат различна структура. Например при някои пратки може да има допълнителни полета като *notes* или *fragile*, докато при други те могат да липсват. Това елиминира необходимостта от създаване на множество таблици или използване на празни (NULL) стойности, както е при релационните бази данни.

NoSQL моделът намалява необходимостта от сложни JOIN операции, тъй като свързаните данни могат да бъдат вложени в един документ. Това води до по-бързо извличане на информация и по-опростена логика на заявките.

Заявките към базата данни в проекта са реализирани чрез MongoDB shell (mongosh), като са използвани стандартни операции за създаване, извличане, актуализиране и изтриване на данни, както и агрегиращи операции за анализ на информацията.

III. Архитектура и реализация

Данните в системата са организирани като JSON документи, което позволява гъвкаво моделиране и лесно разширяване на структурата при необходимост. Основните операции върху базата данни включват създаване на нови записи, извличане на информация, актуализиране на съществуващи данни и изтриване на записи. MongoDB предоставя механизми за ефективно изпълнение на тези операции чрез стандартни заявки и агрегиращи механизми.

Архитектурата на базата данни е съставена от няколко основни колекции, които отразяват ключовите обекти в куриерската система. Данните са организирани така, че да осигурят бърз достъп и минимална зависимост между отделните структури.

Колекция - Customers

Колекцията customers съдържа информация за клиентите на куриерската фирма. Всеки документ включва лични данни като име, телефон, имейл и адрес. Адресът е реализиран като вложен обект, съдържащ улица и пощенски код.

За част от клиентите е налична допълнителна информация за фирма, което демонстрира гъвкавостта на MongoDB при работа с различни структури на данните.

```
{
  "customerId": "CUST001",
  "firstName": "Ivan",
  "lastName": "Petrov",
  "phone": "+359888123001",
  "email": "ivan.petrov@email.com",
  "city": "Sofia",
  "address": {
    "street": "bul. Vitosha 15",
    "postalCode": "1000"
  },
  "isBusinessClient": false
}
```

Колекция Couriers

Колекцията couriers съдържа информация за куриерите, които извършват доставките. Данните включват име, телефон, регион на работа, тип превозно средство и статус на активност.

При някои записи може да има допълнително поле, например причина за неактивност, което показва възможността за разширяване на структурата.

```
{
  "courierId": "COURIER001",
  "firstName": "Petar",
  "lastName": "Ivanov",
  "region": "Sofia",
  "vehicleType": "van",
  "isActive": true
}
```

Колекция Offices

Колекцията offices съдържа информация за офисите на куриерската фирма в различни градове. Всеки документ включва адрес и работно време, като работното време е реализирано чрез вложен обект.

Това позволява ясно и структурирано представяне на информацията.

```
{
  "officeId": "OFF001",
  "city": "Sofia",
  "address": {
    "street": "bul. Vitosha 25",
    "postalCode": "1000"
  },
  "workingHours": {
    "mondayToFriday": "08:00 - 18:00",
    "saturday": "09:00 - 14:00",
    "sunday": "closed"
  }
```

Колекция Payments

Колекцията payments съдържа информация за извършените плащания. Данните включват метод на плащане, сума, валута и статус.

При плащания с карта е налично допълнително поле *transactionId*, което отново демонстрира гъвкавостта на документо-ориентирания модел.

Това позволява ясно и структурирано представяне на информацията.

```
{
  "paymentId": "PAY001",
  "method": "cash",
  "amount": 15.50,
  "currency": "BGN",
  "status": "paid",
  "isCashOnDelivery": true
}  }
```

Колекция Packages

Колекцията packages е основната в системата и съдържа информация за пратките. Всеки документ включва данни за подател и получател, тип доставка, тегло, цена и текущ статус.

Особено важно е полето *statusHistory*, което представлява масив от обекти и позволява проследяване на всяка промяна на статуса на пратката във времето. Това е типичен пример за използване на вложени структури в MongoDB.

```
{
  "packageId": "PKG001",
  "trackingNumber": "TRK2026001",
  "customerId": "CUST001",
  "courierId": "COURIER001",
  "deliveryType": "office-to-office",
  "weightKg": 0.5,
  "price": 15.50,
  "status": "delivered",
  "fragile": false,
  "statusHistory": [
    { "status": "created" },
    { "status": "in transit" },
    { "status": "delivered" }
  ]
}
```

IV. Задължителни CRUD операции

Създаване:

- Примерна заявка за създаване използвайки insertOne (Добавяме само 1 нов запис) Можем да използваме и insertMany ако добавяме повече записи.

```
db.packages.insertOne({
  packageId: "PKG011",
  trackingNumber: "TRK2026011",
  customerId: "CUST001",
  courierId: "COURIER002",
  deliveryType: "office-to-office",
  weightKg: 1.2,
  price: 20.00,
  status: "created",
  fragile: false,
  createdAt: new Date()
})
```

Четене:

- Заявките за четене могат да бъдат филтриране по стойности (find()), търсене по индекс или поле, както и използване на оператори като $gte, $lte, $regex, $or, $in.

```
db.packages.find({ fragile: true })
```

- Пример с $gte и $lte:

```
db.packages.find({
  price: { $gte: 20, $lte: 40 }
})
```

- Пример с $or:

```
db.packages.find({
  $or: [
    { status: "pending" },
    { status: "in transit" }
  ]
})
```

- Пример с $in:

```
db.packages.find({
  deliveryType: { $in: ["office-to-office", "address-to-address"] }
})
```

- Пример с $regex:

```
db.customers.find({
  firstName: { $regex: "Iv", $options: "i" }
})
```

Актуализация:

- Примерни заявки за актулизиране на базата данни като можем да използваме updateOne() или updateMany() ако променя повече записи, както и оператори като $set, $inc, $push, $pull.

- Пример с updateOne() и $set:

```
db.packages.updateOne(
  { packageId: "PKG002" },
  { $set: { status: "delivered" } }
)
```

- Пример с updateMany() и $inc:

```
db.packages.updateMany(
  { deliveryType: "office-to-office" },
  { $inc: { price: 2 } }
)
```

- Пример с updateOne() и $push:

```
db.packages.updateOne(
  { packageId: "PKG002" },
  {
    $push: {
      statusHistory: {
        status: "delivered",
```

```
            date: new Date()
          }
        }
      }
    )
```

- Пример с updateOne() и $pull:

```
db.packages.updateOne(
  { packageId: "PKG002" },
  {
    $pull: {
      statusHistory: { status: "created" }
    }
  }
)
```

Изтриване:

- За заявките за изтриване можем да използваме deleteOne() и deleteMany().
  Пример:

```
db.packages.deleteOne({ packageId: "PKG011" })
db.packages.deleteMany({ status: "cancelled" })
```

V. Допълнителни заявки

- Пример за заявка, използваща агрегиране, групиране или сортиране:

```
db.packages.aggregate([
  { $unwind: "$statusHistory" },
  {
    $group: {
      _id: "$statusHistory.status",
      totalOccurrences: { $sum: 1 }
    }
  },
  { $sort: { totalOccurrences: -1 } }
])
```

- Пример за заявка, която комбинира данни от две различни колекции:

```
db.packages.aggregate([
  {
    $lookup: {
      from: "couriers",
      localField: "courierId",
      foreignField: "courierId",
      as: "courierInfo"
    }
  },
  {
    $unwind: "$courierInfo"
  },
  {
    $project: {
      _id: 0,
      packageId: 1,
      trackingNumber: 1,
      status: 1,
```

```
            price: 1,
            courierName: {
              $concat: ["$courierInfo.firstName", " ",
"$courierInfo.lastName"]
            },
            region: "$courierInfo.region",
            vehicleType: "$courierInfo.vehicleType"
          }
        }
      ])
```

VI. Контрол на достъпа до данните

1. Отваряме файла *mongod.cfg* като администратор за да можем да правим промени. Файлът се намира на път:
   C:\Program Files\MongoDB\Server\8.2\bin\mongod.cfg
2. Маха се коментара на *Security* (тоест #)
3. Под *Security* добавяме: *authorization: enabled*
4. Рестартираме MongoDb
5. Създава се програмно роли
6. Създава се програмно потребители
7. Login

- Заявка за създаване на роля за четене от колекция customers

```
db.createRole({
  role: "customerReader",
  privileges: [
    {
      resource: {
        db: "courierDB",
        collection: "customers"
      },
      actions: ["find"]
    }
  ],
  roles: []
})
```

- Заявка за създаване на роля с права – писане/редактиране от колекция packages

```
db.createRole({
  role: "packageWriter",
  privileges: [
    {
      resource: {
        db: "courierDB",
        collection: "packages"
      },
      actions: ["insert", "update"]
    }
```

```
        ],
        roles: []
```

})

```
      db.createUser({
        user: "employeeReader",
        pwd: "bbbb2222",
        roles: [
          { role: "customerReader", db: "courierDB" }
        ]
      })
```

```
      db.createUser({
        user: "employeeWriter",
        pwd: "aaaa1111",
        roles: [
          { role: "packageWriter", db: "courierDB" }
        ]
      })
```

VII. Използвана литература:

[1] MongoDB Inc. MongoDB Documentation.
https://www.mongodb.com/docs/

[2] Amazon Web Services. Introduction to NoSQL Databases.
https://aws.amazon.com/nosql/

[3] Amazon Web Services. Overview of SQL Databases. Available at:
https://aws.amazon.com/what-is/sql/